

Chapter 7 explain

Chapter 7 Outline

مخطط الفصل السابع

المحتويات الرئيسية للفصل:

- استعلامات SQL أكثر تعقيدًا (More Complex SQL Retrieval Queries)
- تحديد القيود الدلالية كـ **Assertions** والإجراءات كـ **Triggers**
- الـ **Views** (الجدول الافتراضية) في SQL
- تعديل المخطط (Schema Modification) في SQL

✓ ملاحظة مهمة: هذا الفصل يركز على الجوانب المتقدمة من لغة SQL بعد الأساسيات التي تعلمتها في الفصول السابقة.

More Complex SQL Retrieval Queries

استعلامات استرجاع SQL الأكثر تعقيدًا

الشرح:

تقدم SQL ميزات إضافية تسمح للمستخدمين بكتابة استعلامات أكثر تعقيدًا لاسترجاع البيانات من قواعد البيانات. وتشمل هذه الميزات:

- الاستعلامات المتداخلة (Nested Queries)
- الـ **Outer Joins** (الـ **Joined Tables**) والـ **FROM** داخل جملة
- الدوال التجميعية (Aggregate Functions) مثل **COUNT** , **SUM** , **AVG**
- التجميع (Grouping) باستخدام **GROUP BY**

✓ ملاحظة: هذه الأدوات تُستخدم معًا غالبًا لبناء استعلامات قوية تُعيد نتائج دقيقة ومعقدة.

Comparisons Involving NULL and Three-Valued Logic

المقارنات التي تتضمن القيمة NULL والمنطق ثلاثي القيم

الشرح:

القيمة NULL في SQL لا تعني "صفر" أو "سلسلة فارغة"، بل تُستخدم للدلالة على:

1. قيمة غير معروفة (Unknown value)

2. قيمة غير متاحة أو ممنوعة (Unavailable or withheld value)

3. سمة غير منطبقة (Not applicable attribute)

خصائص الـ NULL:

- كل NULL يُعتبر مختلفًا عن أي NULL آخر (لا يمكن مقارنتها بالمساواة).
- SQL (Three-Valued Logic) تستخدم منطقًا ثلاثي القيم:
 - TRUE
 - FALSE
 - UNKNOWN ("مثل" ربما)

⚠ ملاحظة حرجية:

- التعبير `NULL = NULL` لا يُرجع TRUE، بل يُرجع UNKNOWN
- لذلك، لا يُنصح أبدًا بمقارنة القيم باستخدام `NULL =`

Comparisons Involving NULL and Three-Valued Logic (cont'd.)

استمرار شرح المقارنات مع NULL والمنطق ثلاثي القيم

الشرح:

لأن المقارنات العادية لا تعمل مع NULL، توفر SQL طريقة خاصة للتحقق من وجود NULL:

- استخدم:

```
WHERE column_name IS NULL
```

أو

```
WHERE column_name IS NOT NULL
```

✓ مثال:

```
SELECT * FROM EMPLOYEE WHERE Super_ssn IS NULL;
```

يُرجع الموظفين الذين ليس لديهم مشرف (Super_ssn غير معروف أو غير منطبق).

Nested Queries, Tuples, and Set/Multiset Comparisons

الاستعلامات المتداخلة، والـ Tuples، ومقارنات المجموعات/المجموعات المتعددة

الشرح:

- الاستعلام المتداخل (Nested Query):
هو استعلام كامل (SELECT-FROM-WHERE) موجود داخل جملة WHERE لاستعلام خارجي.
- Outer Query = الاستعلام الخارجي
- Subquery = الاستعلام الداخلي
- عامل المقارنة IN :
يستخدم لمقارنة قيمة واحدة v بمجموعة قيم V .
- يُرجع TRUE إذا كانت v موجودة في V .

مثال: ✓

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Ssn IN (SELECT Essn FROM DEPENDENT)
```

يُرجع أسماء الموظفين الذين لديهم معالين.

Nested Queries (cont'd.)

استمرار شرح الاستعلامات المتداخلة

الشرح:

يمكن استخدام الـ **Tuples** (القيم المتعددة) في المقارنات، عن طريق وضعها بين قوسين.

مثال: ✓

```
* SELECT
FROM EMPLOYEE
WHERE (Fname, Lname) IN (SELECT Dependent_name, Sex FROM DEPENDENT)
```

(هذا مثال نظري؛ قد لا يكون منطقيًا دائمًا، لكنه يوضح الفكرة)

Nested Queries (cont'd.) – ANY/SOME و ALL

مواصلة شرح الاستعلامات المتداخلة: عوامل ANY/SOME و ALL

الشرح:

بالإضافة إلى IN ، يمكن استخدام العوامل التالية مع الاستعلامات المتداخلة:

- = ANY أو = SOME :

- تعادل تمامًا IN

- تُرجع TRUE إذا كانت القيمة تساوي أي عنصر في النتيجة
 - العوامل الأخرى مع ANY / SOME :
 > ANY , >= ANY , < ANY , <= ANY , <> ANY
 - مثال: Salary > ANY (SELECT Salary FROM EMPLOYEE WHERE Dno = 5)
 → يُرجع الموظفين ذوي الرواتب أعلى من على الأقل موظف واحد في القسم 5
 - ALL :
 يجب أن تحقق القيمة الشرط مع كل العناصر في النتيجة
 - مثال: Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5)
 → يُرجع الموظفين ذوي الرواتب أعلى من جميع موظفي القسم 5
- ⚠ ملاحظة:
- > ALL (empty set) → يُرجع TRUE (لأن لا يوجد عنصر يخالف الشرط)
 - > ANY (empty set) → يُرجع FALSE

Nested Queries (cont'd.) – Best Practices

نصائح لتجنب الأخطاء في الاستعلامات المتداخلة

الشرح:

لتجنب الغموض أو الأخطاء (خاصة عند وجود أسماء أعمدة مكررة في جداول مختلفة)، يُنصح دائماً بـ:

- إنشاء متغيرات جداول (Table Aliases) باستخدام AS أو كتابة الاسم مباشرة بعد اسم الجدول

✓ مثال:

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN (SELECT D.Essn FROM DEPENDENT AS D)
```

✓ فائدة: تجعل الاستعلام أوضح، وتمنع تضارب الأسماء.

Correlated Nested Queries

الاستعلامات المتداخلة المرتبطة (Correlated Nested Queries)

الشرح:

- الاستعلام المرتبط (Correlated Subquery):
هو استعلام داخلي يعتمد على قيمة من الاستعلام الخارجي.
- يُنفَّذ مرة واحدة لكل سجل (tuple) في الاستعلام الخارجي.

- يمكن أحياناً تحويل الاستعلام المتداخل المرتبط إلى استعلام باستخدام JOIN.

✓ مثال من الشرائح (Q16):

الشكل المتداخل:

```
SELECT Fname, Lname
FROM EMPLOYEE
) WHERE Ssn IN
SELECT Essn
FROM DEPENDENT
WHERE Sex = EMPLOYEE.Sex
AND Dependent_name = EMPLOYEE.Fname
;
```

الشكل المكافئ باستخدام JOIN (Q16A):

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn
AND E.Sex = D.Sex
; AND E.Fname = D.Dependent_name
```

⚠ ملاحظة مهمة:

- ليس كل استعلام متداخل يمكن تحويله إلى JOIN
- الاستعلامات المرتبطة قد تكون أبطأ لأنها تُنفَّذ بشكل متكرر

ملخص استراتيجي للمراجعة:

المفهوم	الكلمة المفتاحية	الاستخدام
NULL	IS NULL / IS NOT NULL	للتحقق من القيم المفقودة
Nested Query	IN , ANY , ALL	مقارنة قيمة بمجموعة نتائج
Correlated Query	يحتوي على مرجع للـ Outer Query	يُنفَّذ لكل سجل خارجي
Tuple Comparison	(col1, col2) = (val1, val2)	مقارنة أزواج قيم
Alias	أو اسم بعد الجدول AS	لتجنب الغموض